Production breakdown: 讲讲你是怎样作一首曲子的

Oct 22, 2026 · @txul16Et

免费分享声明 · 本文档由本项目作者免费分享，仅供个人学习与非商业使用。禁止以任何名义出售、转售或打包售卖，禁止通过付费下载、付费会员、付费课程、付费社群、“打赏/关注后获取”等任何形式收费。转发须注明出处并保留本声明。许可：CC BY-NC-SA 4.0。

Free-sharing notice · Shared free for personal, non-commercial use. Selling it, or charging for access in any form or under any pretext, is prohibited. Share with credit and keep this notice. Licence: CC BY-NC-SA 4.0 (https://creativecommons.org/licenses/by-nc-sa/4.0/).

Overview

The film is a 4 min 52 s answer to one question, "讲讲你是怎样作一首曲子的": instead of explaining, the AI writes the piece the viewer is hearing, and the picture shows that piece being built note by note, then growing into a 3D landscape and folding back into the project.

免费分享 · 禁止售卖 · FREE TO SHARE · NOT FOR SALE

Item	Value
Running time	292.4 s (85 bars plus a free intro and tail)
Picture	1920×1080, 30 fps, 8,773 frames
Music	Original score in D major, lifting to E major at bar 57; base tempo 80 BPM with a 60→44 BPM free intro and a 74→50 BPM final ritardando
Ensemble	Piano, cello, double bass, drum kit, string section and violin, horn, flute, clarinet, glockenspiel, harp, choir, timpani
Language	All on-screen text in Chinese (55 caption cues); code keeps its own syntax
Guide character	The original pixel-art Claude desktop pet, unmodified, split into one collaborator per track
Visual identity	Near-black background, warm coral accent (RGB 217,119,87), a colour per voice, thin luminous lines

How it is built. Everything comes from code and from one source of truth:

1. `score.py` writes the whole score as data: notes, tempo map, chords and expression curves.
2. `render.c` and `mix.py` play that score through the GeneralUser GS SoundFont and mix the result; the chat typing and send sounds are mixed in too.
3. The same note list is exported to the renderer (`vis5.json`), so every note on screen sits exactly where it sounds.
4. `video.html` draws each frame as a deterministic function `renderAt(t)` , in 2D canvas with a WebGL bloom pass; `journey.js` adds the 3D multitrack journey and `world3d.js` the 3D landscape.
5. Playwright drives Chrome frame by frame; ffmpeg encodes the frames and adds the audio.

Prompt compilation

The project went through 12 rounds of direction: an initial concept, two long written briefs, then a run of revisions that moved from storytelling to 3D craft to render stability. Two briefs survive word for word as attachments; most later prompts are quoted from the conversation record, and the very first concept prompt survives only as a summary.

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

#	Prompt (quoted where the wording survives)	Text	What it changed
1	Initial concept: an AI composes a song that tells the story of its own creation; musical quality is the top priority	Summary only	Set the premise, the score-as-code approach and the first full arrangement
2	Brief A, visual direction (full text below)	Verbatim	Kept the music; added the desktop pet as active creator, a visual build that follows the arrangement, the notes-become-a-world climax, the return loop, and code-to-result moments
3	Brief B, caption and storytelling revision (full text below)	Verbatim	Rewrote the full Chinese caption script; rebuilt note display on one shared time axis; one focus per creative stage; one landscape transformation at a time
4	Four revisions: more variety in the multitrack section with a continuous camera; the reply becomes "收到。" then "但是，与其给你讲解原理，不如直接写一首曲子。"; a more spectacular world; an ending that rewrites the melody in close-up, then pulls back and packages the piece	Paraphrased, captions verbatim	Produced the 3D multitrack journey, the new opening reply, and the close-up rewrite plus chat-card ending
5	"In the sequence where the music forms an entire world, selectively incorporate 3D techniques… refined materials and lighting. Avoid a cheap, plastic-looking aesthetic."	Verbatim, trimmed	Moved the landscape into true 3D (WebGL) with soft shadows, fog and reflections

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

#	Prompt (quoted where the wording survives)	Text	What it changed
6	Redesign of the world: clarify or remove the circle/ellipse; make the rivers convincing; build volumes from families of parallel lines; one continuous camera journey through rivers, lakes and sea; keep the music–landscape link	Paraphrased	Rebuilt the world from the harmony's 5 staff lines (the river) and the melody (the ridges), with a fly-through camera
7	"You can use GPU acceleration to speed up rendering."	Verbatim	Led to the portable local GPU render package
8	"During the final camera movement through the mountains, rivers, lakes, and sea, the bottom of the frame appears to jitter. Can you confirm whether this is an actual rendering issue?"	Verbatim	Diagnosed temporal aliasing on the water lines
9	"Please check the final camera transition from the 3D world back to the music tracks. The camera rotation appears to stutter during this transition. While rendering continues, please also sync the rendering project and all required assets to my computer, and provide a script to launch the render."	Verbatim	Smoothed the return camera; delivered render_project and 开始渲染.bat

免费分享 • 禁止售卖 FREE TO SHARE • NOT FOR SALE

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

#	Prompt (quoted where the wording survives)	Text	What it changed
10	"The lake surface, small boat, and other elements in the 3D scene still jitter from side to side and back and forth, with visible flickering in the rendered output. The motion lacks continuity and smoothness. The camera movement at the end still feels discontinuous and jerky."	Verbatim	Fixed raft position jumps and the camera's yaw stall; added a world-space glide for the return
11	"Stop the rendering currently running on your cloud computer."	Verbatim	Cloud render stopped; local GPU render became the main path
12	"The lake surface still exhibits severe side-to-side jitter. This issue needs to be resolved at its root… the final camera transition from the 3D world back to the music tracks still feels abrupt and disjointed."	Verbatim, trimmed	Root-cause fix: per-pixel analytic water plane; reworked hand-off from the world back to the tracks

Short status prompts ("What's the current progress?", "Retry.") are left out of the table.

Brief A — visual direction (verbatim)

```
The musical arrangement is already working well. Please preserve the existing
structure, melody, instrumentation, and emotional progression, and focus on
improving the visuals, animation, and editing.
The video's central premise is: “Tell me how you compose a piece of music.”
The entire video is the AI’s response, and the piece playing throughout is
the response itself. As the music develops, the visuals should reveal how it
is being created, bringing the creative process and the finished work
together.
Use the provided reference video as visual inspiration: a dark background,
warm coral-orange accents, refined line animation, a pixel-art Claude desktop
pet, and a narrative that expands from small interface interactions into a
complete world before returning to the original interface. You should lead
the detailed visual storytelling, guided by the following requirements:
1. Use Chinese subtitles throughout
```


免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

All subtitles, titles, explanatory text, and narrative interface labels must be in Chinese. Preserve authentic syntax in code snippets. Keep typography clean and readable, introduce text in time with musical phrases, and allow sufficient reading time. Avoid lengthy written explanations of the music.
2. Let the Claude desktop pet actively create the music
Add animations of the Claude desktop pet manipulating tracks, writing notes, adjusting rhythms, moving melodic phrases, and drawing automation curves. Its actions should have a clear purpose and correspond to audible changes: the melody plays as notes are placed, percussion enters as rhythmic patterns are arranged, and the sound becomes richer as harmony tracks unfold.
Make the pet the creator and visual guide throughout the video. It may move between tracks or split into multiple collaborators as different musical parts develop, but its actions should remain connected to the music.
3. Let the visuals grow with the music
Begin with restraint: a short melodic idea, a few notes, and a minimal composition interface. As the existing arrangement develops, gradually introduce rhythm, bass, harmony, and additional parts, building visual complexity alongside the music.
Editing, camera movement, and transitions should follow musical phrasing, breathing space, instrumental entrances, and emotional shifts. Preserve negative space in quieter passages, gradually add visual layers during the build, and release the full visual imagination at the climax. Avoid constant visual density or mechanically cutting on every beat.
4. At the climax, let the notes grow into a living world
As the music reaches its climax, notes and melodic lines should gradually detach from their tracks, becoming fine, luminous strands that extend beyond the composition interface into a larger space.
Through continuous, visually traceable transformations, these strands and notes should form imagery such as the night sky, stars, the moon, mountains, rivers, lakes, and oceans. For example, an extended melodic line might curve into a mountain ridge, unfolding layers of harmony might form the surface of water, and small notes might rise into points of starlight. Design the specific relationships according to the character of the music.
Make it clear that the world on screen has grown out of the tracks and notes the audience saw earlier. Retain recognizable elements of notation, waveforms, beats, or track lines within the landscapes, maintaining a visible connection between nature and music. The sequence should feel poetic and alive. Avoid inserting disconnected scenic footage.
5. Complete the visual loop at the ending
As the music resolves, the sky, moon, mountains, and water should gradually transform back into notes and strands, following recognizable paths into the composition interface and returning to their original tracks.
End with the opening question, the Claude desktop pet, and the completed music project. The audience should recognize that the entire world they just experienced emerged from this piece of music—and from that first sentence.
6. Show how code constructs both the music and the visuals

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

At key moments, show clear cause-and-effect relationships between code and its results: code defines notes, rhythms, instrument parameters, and animation behavior; when it executes, the corresponding musical structures and visuals take shape. Code fragments may transition into notes, tracks, or lines within the scene to create natural visual continuity.
Keep code displays concise, credible, and meaningful. Avoid meaningless scrolling characters used merely to suggest technology. Help the audience understand how code participates in the creative process while preserving the music's emotion and the visuals' beauty.
The final goal is to create a music piece that is enjoyable in its own right, while allowing the audience to naturally understand—through listening and watching—how AI uses code to compose the piece and build the visual world that grows alongside it.

Brief B — caption and storytelling revision (verbatim)

Please revise the existing video, **"告诉我，你是怎么作曲的"**, using the instructions below. Preserve the current musical composition, arrangement, instrumentation, duration, and emotional progression. Focus on rewriting the Chinese captions and improving visual storytelling, animation, editing, and the clarity of the musical interface.

The premise is: **the AI answers a question about composing by composing the very piece the audience is hearing.** The visuals should reveal the creative process as the music unfolds.

1. Visual style and original Claude desktop pet

Use the reference video's restrained visual language: dark backgrounds, warm coral-orange accents, clear typography, precise line animation, and deliberate negative space.

Use the original Claude desktop pet exactly as shown in the supplied screenshot. Preserve its silhouette, proportions, pixel structure, face, and original colors. Do not redesign, reinterpret, accessorize, or turn it into a music-themed character. Do not add musical notes, hats, instruments, or other features to its body.

Animate the original character through position, orientation, and its existing animation states. Any interaction effects must remain separate from the character's design.

Make the pet an active creator. Show clear sequences of **intention → action → visible result**: placing a note, moving its pitch, extending its duration, arranging a rhythmic pattern, handing a phrase to another voice, or adjusting an expression curve. Avoid making it merely follow the playhead or bounce

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

```
beside notes.

### 2. Opening sequence

Begin with the question:

**“告诉我，你是怎么作曲的？”**

The AI responds through the following Chinese captions. Display each caption
separately, allowing enough reading time:

> 与其讲解怎样作曲，
> 不如，我直接写一首给你听。
>
> 接下来你听到的音乐，
> 没有人坐在钢琴前，
>
> 也没有乐队在录音棚里演奏。
>
> 有的，只是一行行代码。
>
> 我用它写下音符，
> 安排节奏，
>
> 让不同的声音，
> 在时间里相遇。
>
> 先从一个音符开始。

Suggested visual progression:

- Begin in the conversation interface. The pet moves from the response into a
minimal composition workspace.
- Reveal a fine timeline in the empty workspace.
- Briefly introduce simple outlines of a piano and an empty piano bench, then
let them fade.
- Introduce outlines of an ensemble and recording microphones, then let them
recede.
- Reveal a short, meaningful code snippet.
- The pet executes the code. Highlight the pitch and duration values as they
generate the corresponding note.
- Move closer as the pet places the first note on the track.

Fit this sequence into the existing musical opening. Do not extend, delay, or
rearrange the music simply to accommodate the text. If necessary, streamline
caption transitions while preserving comfortable readability.
```

免费分享 • 禁止售卖 • FREE TO SHARE • NOT FOR SALE

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

3. Correct the musical timeline and note visualization

Rebuild the note display to eliminate misaligned timing, unintended intersections, overlapping graphics, and inconsistent positioning.

Use the underlying musical event data as the single source of truth for note placement and playback:

- All tracks must share one horizontal time axis, tempo map, and scrolling transformation.
- Notes that begin simultaneously must align vertically across tracks.
- A note’s horizontal position must represent its onset, its width must represent its duration, and its vertical position must consistently represent pitch within its track.
- Align bar lines, beat divisions, note positions, and the playhead. They must not drift relative to one another.
- Highlight a note when its audible onset reaches the playhead, and sustain its active state for the corresponding duration.
- Keep simultaneous chord tones horizontally aligned and vertically separated.
- Preserve genuine musical overlaps and polyphony, but distinguish separate voices clearly. Do not “fix” valid music by changing note timing.
- Prevent notes, labels, pets, code panels, and automation curves from unintentionally covering one another.
- Put automation curves in clearly separated lanes or use a distinct visual treatment so they cannot be mistaken for note data.
- When zooming or scrolling, transform the grid, notes, playhead, labels, and interaction anchors coherently.
- Give percussion a legible rhythmic display rather than tiny, nearly invisible dots.
- Use a consistent, refined visual vocabulary for notes. Piano-roll bars are appropriate in the composition interface; abstract strands should emerge during the later transformation sequence.

At the transition into the landscape, deliberately release notes from the timeline. Until that moment, maintain precise musical alignment.

4. Revised Chinese caption script

All audience-facing captions, titles, explanatory labels, and narrative interface text must be in Chinese. Code may retain authentic syntax and identifiers.

Use the following script after the opening sequence. Match captions to the corresponding musical events rather than forcing them into arbitrary time

免费分享 · 禁止售卖 · FREE TO SHARE · NOT FOR SALE

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

intervals. Each paragraph below is a separate caption cue.

[The full caption script is reproduced, as finally timed, in the storyboard section of this document.]

5. Redesign the visual progression

The current video spends too long in an almost unchanged multitrack view. Preserve the music, but give each creative stage its own visual focus:

- **Melody:** move close to individual notes and show how pitch and duration shape a phrase.
- **Variation:** visually relate the original phrase to its modified version, making the change easy to recognize.
- **Harmony:** reveal vertically aligned chord tones beneath the melody. Emphasize the specific chord being introduced or changed.
- **Bass:** shift attention downward and show the relationship between bass movement and harmony.
- **Rhythm:** introduce a clear beat grid and purposeful rhythmic placement.
- **Second voice:** show a recognizable phrase passing between two voices, with the pets performing a visible handoff.
- **Instrumentation:** connect instrument labels and track colors to the phrase being played. Make the transfer between timbres visible.
- **Full ensemble:** pull back to reveal the complete arrangement only after the audience understands its parts.

Alternate close-ups, focused track views, and full-project views according to musical phrasing. Avoid arbitrary camera movement, constant cutting, or prolonged static compositions.

6. Make the transformation continuous and readable

At the musical expansion, let notes leave the tracks and become a landscape through clear, traceable transformations.

Establish one transformation at a time:

- A recognizable melodic contour extends into a mountain ridge.
- Layers of harmony spread into the water’s surface.
- Short, bright notes rise into stars.
- A continuing melodic line flows through the valley as a river.
- The first note grows into the moon.

Arrange these events in an order that fits the existing music and caption sequence. Retain each source element’s identity through color, contour, or motion so the audience can follow where it goes.

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

Avoid a sudden burst of tangled particles that conceals the transformation. Do not let code panels cover the focal action.

Once the world is established, develop the scene through purposeful camera movement and evolving details. Follow a musical line through the landscape instead of holding the same wide view for an extended period.

7. Show meaningful code-to-result relationships

At selected moments:

Highlight a short code instruction → execute it → show the corresponding musical or visual change → clear the panel.

Use concise, credible code displays. Each appearance should explain one relationship. Avoid simultaneous multi-line code, dense animation, moving pets, and long captions competing for attention.

The world-building code should visibly correspond to the mountain, water, stars, or moon being formed at that moment.

8. Complete the ending coherently

As the music resolves, transform the landscape back into its identifiable source notes and tracks. Keep the return paths readable and preserve the identity of the musical parts.

Reveal the completed project, briefly emphasize the original melodic phrase within it, then return to the conversation interface.

The caption script intentionally preserves the full arrangement at the ending. Do not claim that only a seed or a single note remains while the screen still shows every track.

9. Caption readability and final review

- Display one caption cue at a time, with no more than two lines.
- Use consistent Chinese typography, punctuation, size, contrast, and placement.
- Keep captions inside a safe area and separate from track labels and controls.
- Leave deliberate text-free passages, especially during the full ensemble and landscape sequence.
- Use complete Chinese characters in finished captions. Any simulated Chinese input method animation must resolve cleanly before the scene changes.

免费分享 · 禁止售卖 · FREE TO SHARE · NOT FOR SALE

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

```
- Check readability at ordinary embedded-player and mobile viewing sizes.
- Verify that every caption describes what is actually visible or audibly
happening.
- Review note alignment and animation frame by frame around entrances, zooms,
scrolling transitions, and the landscape transformation.

The finished video should feel like **a composition bringing itself into
existence**: understandable at the beginning, increasingly expressive as the
arrangement grows, and poetic when the music becomes a world.
```

One bracketed line in Brief B replaces the brief's own caption list, which appears in full, with its final timings, in the storyboard below.

Storyboard

The film has 9 sequences that follow the score's 10 musical sections exactly; each visual change starts on a bar line taken from the score's tempo map, so no cut or camera move is timed by eye.

#	Time	Bars	Music	Picture	Camera
1	0:00–0:08	before bar 1	Silence, typing clicks, send sound	Chat window; the question is typed with a Chinese input-method animation and sent; the pet replies "收到。"	Static, full chat window
2	0:08–0:25.6	1–4	One sustained note, free tempo 60→44 BPM	Chat opens into an empty workspace; a fine timeline draws in; piano and empty bench outlines fade; ensemble and microphone outlines recede; a code snippet runs and its pitch and duration values become the first note	Glides from the chat to the top-left of the workspace (scale 1, centre 1080,330)
3	0:25.6–0:49.7	5–12	Piano melody alone, 80 BPM; phrase ends on a question (80→70 BPM)	The pet places each note; rising and falling contour drawn as it plays	Close-up on the melody lane (scale 1.6)

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

#	Time	Bars	Music	Picture	Camera
4	0:49.7–1:13.8	13–20	Harmony answers: piano chords	Variation shown against the original phrase; chord tones stack vertically under the melody; the chord being changed is highlighted	Eases out to 1.35, then 1.45 lower to take in the harmony lane
5	1:13.8–2:01.8	21–36	Bass (cello and double bass), drums from bar 29, second melody on cello from bar 29	Bass lane opens and the view drops to it; beat grid and drum rows appear; two pets hand a phrase between voices	Steps down the project lane by lane (scale 1.3 → 1.08)
6	2:01.8–2:25.2	37–44	Pre-chorus rises into the chorus: strings, expression curve, colour instruments, winds	Instrument labels light up with their phrase; the expression curve is drawn in its own lane	Moves between instrument lanes (scale 1.15 → 1.05), then pulls back to the whole project (scale 1)
7	2:25.2–3:09.6	45–60	Chorus end; bridge takes the song apart; breakdown in silence, key lifts to E	3D multitrack journey: the flat project tilts into depth sheets, curls into rings, blooms into a mandala, goes hushed, then lays flat again	Continuous 3D orbit, no cuts
8	3:15.5–4:04.4	61–76	Final chorus, everything at once; verse melody returns as counterpoint	Notes leave the tracks: melody becomes ridges, harmony becomes water and a lake, bright notes become stars, the second melody becomes a river, the first note becomes the moon	Top-down onto the staff, then a low fly-through along the river to the sea and up
9	4:04.4–4:52.4	77–85	Coda, letting go; final ritardando 74→50 BPM	The world flows back into the tracks; the full project is shown; the opening melody is rewritten in close-up; the piece is packaged into a card in the chat	Return glide to the flat project, close-up (scale 1.75), then a pull-back to the chat (scale 1/6)

Caption cues by sequence (55 in all, one at a time, at most two lines):

Time	Caption
0:03.9	收到。
0:05.1	但是，与其给你讲解原理，不如直接写一首曲子。

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

Time	Caption
0:08.1	接下来你听到的音乐，没有人坐在钢琴前，
0:12.0	也没有乐队在录音棚里演奏。
0:16.0	有的，只是一行行代码。
0:19.2	我用它写下音符，安排节奏，
0:22.5	让不同的声音，在时间里相遇。
0:25.3	先从一个音符开始。
0:28.8	音符记下声音的高低，也记下它停留的时间。
0:33.2	把不同高低、不同长短的音依次排列，便有了旋律。
0:38.0	我让它向上，试着靠近期待。
0:43.8	再让它落下，留一点回望。
0:49.8	一段旋律需要被记住，也需要一点变化。
0:55.0	于是，熟悉的几个音，又走出了一条新的路。
0:59.2	旋律之外，还有可以同时响起的声音。
1:03.0	几个音叠在一起，组成和弦。
1:06.8	和弦彼此连接，让音乐有了明暗。
1:14.2	同一段旋律，换一组和弦，心情也会不同。
1:20.0	我在明亮的地方，留下一点温柔的阴影。
1:26.0	再添上一行低音，托住和声，也引向下一步。
1:30.6	让旋律走得再远，脚下也有归处。
1:34.8	音符的长短、轻重与停顿，让音乐有了节奏。
1:38.0	鼓点加入，把前行的脚步变得清晰。
1:42.4	我再写一条旋律，让它与第一条相遇。
1:47.0	有时同行，
1:56.0	有时，接过对方未尽的话。
2:02.0	接下来，为声音选择音色。
2:05.6	弦乐把呼吸拉长，
2:14.1	同一句旋律，交给不同的乐器，会有不同的神情。

免费分享 • 禁止售卖 • FREE TO SHARE • NOT FOR SALE

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

Time	Caption
2:19.0	管乐从间隙中轻轻回应。
2:26.8	哪个声音先来，哪个声音退后，
2:30.4	哪里相拥，哪里留白。
2:33.2	这些安排，让旋律逐渐长成完整的作品。
2:38.0	我让它换一种声音，再讲一次。
2:50.0	当所有声部相遇，
2:53.0	最初的几个音，已经有了辽阔的回声。
3:02.2	也让它安静片刻，
3:05.4	给下一次展开，留出空间。
3:09.0	这些音符、节奏与相遇，都被我写进代码。
3:12.6	接下来，我也想让你看见它们。
3:16.0	音符沿着旋律，走出了音轨。
3:20.6	起伏的旋律，绵延成山。
3:26.0	层叠的和声，铺开成水。
3:31.4	细碎而清亮的音，升起，成为星光。
3:40.0	那句还未结束的旋律，沿着山谷，流向远方。
3:45.8	最初的一个音，在夜空中，亮成了月亮。
3:55.0	你听见的声音，在这里，有了形状。
4:04.8	乐句渐渐收拢。
4:08.4	山川、流水与星光，沿着来路，回到音轨。
4:13.8	每一个声部，都留下了自己走过的路。
4:18.6	而最初的旋律，依然在它们之间。
4:34.2	这就是我作曲的过程。
4:37.6	从几个音开始，
4:40.4	让它们彼此连接，慢慢成为一个世界。

Text-free passages are deliberate: 2:57.8–3:02.2 (full ensemble peak), 3:35.1–3:40.0, 3:50.0–3:55.0 and 3:59.4–4:04.8 (landscape), 4:23.0–4:34.2 (close-up rewrite and packaging).

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

Scene descriptions

Every scene follows the same rule from Brief B: intention, then action, then a visible and audible result. A code card shows one instruction, the pet runs it, the music or the picture changes, and the card clears.

1. The question (0:00–0:08)

A chat window on a near-black page. The question is typed through a Chinese input-method animation: pinyin candidates resolve into 讲讲 / 你是 / 怎样 / 作一首 / 曲子的, each with a key-click in the mix (typing starts at 0.55 s, sent at 3.2 s). The pet appears as the reply bubble and says "收到。", then "但是，与其给你讲解原理，不如直接写一首曲子。"

2. No one at the piano (0:08–0:25.6, bars 1–4)

The chat dissolves into an empty workspace with a fine timeline. Three short code cards drive the scene:

Time	Code on screen	Result
0:07.6	`sketch(Grand(keys=88, lid=35))`	A plotting pen draws a grand piano and an empty bench, which fade
0:11.8	`sketch(Ensemble(players=3, drums=1, mics=2))`	Outlines of players, a drum kit and two microphones, which recede
0:15.9–0:23.6	`seed = ["A4", "D5", "E5", "F#5"]`, `rhythm = [0.5, 0.5, 1, 1]`, `melody.add(pitch="A4", beats=0.5)`	The pitch and duration values light up; a ghost note leaves the code and the pet writes it onto the melody track

The music here is one sustained note over a free, slowing tempo (60→44 BPM), so the captions have room to breathe without the music being stretched.

3. Melody (0:25.6–0:49.7, bars 5–12)

Close on the melody lane. The pet places each note as it sounds; height is pitch and length is duration, exactly as the caption says. The phrase rises ("我让它向上") and falls ("再让它落下"), and the tempo eases 80→70 BPM so it ends like a question.

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

4. Variation and harmony (0:49.7–1:13.8, bars 13–20)

The original phrase is shown beside its variation so the changed notes read at a glance. Chord tones then stack vertically under the melody; the chord being introduced or changed is highlighted. Each harmony caption is timed to the chord change it describes.

5. Bass, rhythm and a second voice (1:13.8–2:01.8, bars 21–36)

The bass lane (cello and double bass) opens and the camera drops to show bass moving under the chords. At 1:35 a drum card runs `kick = "x . . . . x . ."` and `stick = ". . x . . . x ."`; the pattern lands on a five-row drum grid (镲, 通鼓, 沙锤 · 镲, 军鼓, 底鼓) and the drums enter on bar 29. A second pet writes the cello counter-melody; the two pets hand a phrase back and forth ("有时，接过对方未尽的话").

6. Colour of sound (2:01.8–2:25.2, bars 37–44)

Instrument labels light up with the phrase they play. A card runs `strings.expression.ramp(30, 105)` and an expression curve is drawn in its own lane, clearly apart from note data. Winds answer from the gaps; the camera then pulls back to the full nine-lane project for the first time.

7. The 3D multitrack journey (2:25.2–3:15.5, bars 45–60)

The whole project lifts into 3D and stays one continuous shot:

1. **Flat** — the normal project view.
2. **Depth sheets** (from 2:25.6) — each track becomes a sheet at its own depth (melody nearest, drums farthest).
3. **Ring** (from 2:36.6) — the sheets curl into concentric rings; melody on the outer ring (radius 800), drums at the centre (160); notes rotate past a fixed playhead.
4. **Mandala** (from 2:48.4) — at the full-ensemble peak the rings bloom into a radial pattern; notes lift on their attack.
5. **Hush** (from 3:01.4) — the breakdown; the pattern shrinks and dims as the key lifts to E.
6. **Flat again** (from 3:06.2) — back to the project, ready for `score = [melody, chords, bass, drums, voice2, strings, winds, colors]` and `world = render(score)`.

8. The world (3:15.5–4:04.4, bars 61–76)

The notes leave the tracks one family at a time, each keeping its track colour, and each family arrives with its own one-line code card:

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

Time	Code	Source	Becomes
3:19.8	`ridge = contour(melody)`	Melody (coral)	Mountain ridges traced from the melody's pitch contour
3:25.4	`water = spread(harmony)`	Harmony (sand); bass (deep red) for depth	Water surface and a lake; bass notes form the depths
3:30.8	`stars = rise(short_notes)`	Winds (amber); drums give glints	Stars; percussion glints on the water
3:39.4	`river = flow(voice2)`	Second melody (teal)	A river through the valley
3:45.2	`moon = grow(first_note)`	The very first note	The moon

The landscape is true 3D: the river is built from the harmony staff's five lines, mountains and water are drawn as families of parallel lines, and the pets ride a small raft down the river. The camera starts looking straight down onto the staff, tips forward into a low fly-through along the river, reaches the lake and the sea, then rises (detail in the camera section).

9. Return, rewrite and package (4:04.4–4:52.4, bars 77–85)

The world flows back along its own paths in reverse order: stars, river, water and lake, ridges, then the moon last (4:05.4–4:11.4). The camera glides back to the flat project and the pets reappear on their tracks. Close on the melody, a card runs `seed = ["B4", "E5", "F#5", "G#5"]` and `melody.add(seed)`: the opening motif, now in E, is rewritten as it plays. Then `project.export("告诉我，你是怎么作曲的.mp4")` sweeps a packaging bar across the project (打包中 → ✓ 已打包, "音乐 · 画面 · 8 条音轨 · 85 小节"). The camera pulls back until the whole project is a card in the original chat. The last captions play, and the title card closes the film: "告诉我，你是怎么作曲的" / "一首曲子，讲述自己的诞生。" with the pet above.

Animation, camera and transitions

The film is one continuous shot with no hard cuts: every change of space is a camera move or a transformation, and each one starts on a phrase boundary from the score.

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

Camera

Part	How it moves	Timing rule
2D workspace (0:00–2:25)	26 keyframes of scale and centre, eased in and out (cubic); 2–3 s moves, then holds	Moves start about 2 s before the phrase they frame; holds cover the phrase
3D journey (2:25–3:15)	A virtual camera orbits the arranged tracks; five stage blends of 2.6–3.8 s each	Stage changes on section entrances (chorus end, bridge, peak, breakdown)
3D world (3:17–4:13)	22 keyframes of path position, height, side offset, yaw, pitch and lens, joined by a shape-preserving cubic (PCHIP) so speed changes smoothly and never overshoots; then a temporal Gaussian smoothing pass	Each landscape element is introduced while the camera frames it
Return home (4:06–4:13)	A separate world-space glide replaces the path-following camera: heading turns once, roll is held level, and a 2.1 s blend hands over	Lands back on the flat project as the coda's first phrase ends
Ending (4:23–4:34)	Close-up to scale 1.75 on the melody, then a pull-back to scale 1/6 so the project becomes a card in the chat	Pull-back starts as the export bar finishes

The world camera's path through the landscape, by key: looking straight down onto the staff (3:17.2) → tips forward to just above the river (3:20.9) → low along the river, drifting side to side through the bends (3:23–3:42) → out over the lake and the sea (3:45–3:48) → climbs to 480 units and turns back (3:54–4:04) → rotates through 135° while descending (4:04–4:09) → straight down onto the staff again (4:11.6).

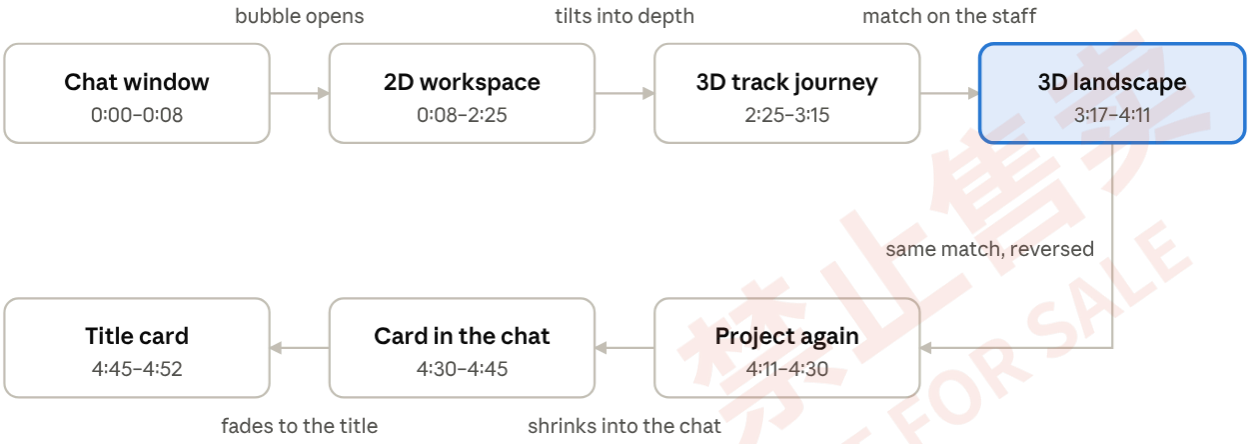
Transitions

- **Chat → workspace**: the reply bubble opens into the workspace while the pet walks from its message into the timeline.
- **Workspace → 3D journey**: the flat project tilts into depth; the playhead stays in place so timing never jumps.

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

- **Journey → world**: the camera zooms to scale 3 on the harmony staff (3:17.9); the 3D world starts from the matching top-down view of the same five lines, so the cut is invisible.
- **World → tracks**: the same match in reverse; the landscape returns along its own paths, the camera arrives top-down on the staff (4:11.6) and pulls out of scale 3 to the full project (4:15.6).
- **Project → chat**: the packaged project shrinks into the chat as a message card.

The film travels out to a 3D world and back, without a cut



the film's spaces and how each hands over to the next

The outward path and the return use the same match point on the harmony staff, so the loop closes where it opened.

Animation principles

- **One relationship at a time**: each code card holds one instruction; values light up, a ghost of the result leaves the card, then the card fades (0.35 s in, 0.45 s out).
- **The pet stays the original**: pixel map copied from the supplied screenshot; it acts through position, facing and its existing states only. It splits into one collaborator per track and rides a raft through the world.
- **Notes live on the score**: a note lights when its sound starts and stays lit for its length; in the journey, notes lift with their loudness and settle over about 1 s.
- **Identity by colour**: each voice keeps its colour from track to landscape (melody coral, harmony sand, bass deep red, drums ivory, second melody teal, strings rose, winds amber, colour instruments blue).
- **Light**: a WebGL bloom on bright notes, a subtle lens fringe, vignette and film grain; bloom follows the music's loudness.

免费分享 • 禁止售卖 • FREE TO SHARE • NOT FOR SALE

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

Music and audiovisual sync

Picture and sound share one score file, so sync is exact by construction: the renderer reads the same note times the synthesiser played.

The score's form

Section	Bars	Starts	Tempo (BPM)	What it adds
1. One note	1–4	0:08.0	60 → 44, free	A single note
2. A melody	5–12	0:25.6	80, easing to 70	Piano melody alone, ending on a question
3. Harmony	13–20	0:49.7	80, easing to 72	Piano chords, the answer
4. Foundation and pulse	21–36	1:13.8	80	Bass, then drums and a cello second melody from bar 29
5. Rising	37–40	2:01.8	80, easing to 77	Strings, expression swell, colour instruments
6. The song	41–48	2:13.8	80	Full chorus with winds
7. Taking it apart	49–56	2:37.8	80, easing to 70	Bridge that thins the parts
8. Silence	57–60	3:02.0	70, pushing to 80	Breakdown; key lifts from D to E
9. Everything at once	61–76	3:15.5	80, broadening to 66	Final chorus; the verse melody returns as cello and horn counterpoint
10. Letting go	77–85	4:04.4	74, slowing to 50	Coda and final ritardando

Sync method

1. `score.py` holds a tempo map of linear ramps; every note's start and end time is integrated from it, so ritardandos and accelerandos are exact.
2. The same note list, bar times and beat times are written to `vis5.json` (renderer data) and to a Standard MIDI file.

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)

3. Every visual timing is a bar or beat time plus a deliberate offset: track lanes open 2 s before their instrument enters, code cards appear about 0.6 s before they run, and camera moves start about 2 s before the phrase they frame.
4. A note lights on the frame of its own start time; at 30 fps any visual event is at most one frame (33 ms) after its sound.
5. The opening's typing clicks and send sound are placed in `mix.py` at the same times the chat animation uses (typing from 0.55 s, sent at 3.2 s).

Sound sources

The notes are composed in code, and the instrument sounds come from one free sample library: the GeneralUser GS SoundFont (v2.0.3, by S. Christian Collins; free for private and commercial music). `render.c`, built on the TinySoundFont library (MIT licence), plays each of the 16 channels through that SoundFont into its own stem. `mix.py` then applies per-track gain, pan and filtering, a synthetic hall reverb, the typing and send clicks, bus compression and a peak limiter. No recordings of existing music are used, and re-running the chain reproduces the final audio bit for bit.

Channel	Part	GM program in GeneralUser GS
0, 1	Melody, harmony	0 Acoustic Grand Piano
2, 6	Bass line, second melody	42 Cello
3	Contrabass	43 Contrabass
4	Strings	49 Slow Strings
5	Hook	48 String Ensemble
7	Flute	73 Flute
8	Clarinet	71 Clarinet
9	Drums	Standard drum kit
10, 15	Horns, counter-horn	60 French Horn
11	Glockenspiel	9 Glockenspiel
12	Harp	46 Orchestral Harp
13	Timpani	47 Timpani
14	Choir	52 Choir Aahs

免费分享 · 禁止售卖 · FREE TO SHARE · NOT FOR SALE

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

Rendering and delivery

A full render on your own GPU takes about 13 minutes; the same render on the cloud machine (no GPU, 2 CPU cores, software WebGL) took hours and had to be split into resumable chunks.

Stage	Tool	Detail
Frame drawing	Chrome via Playwright	`renderAt(t)` per frame; 2D canvas for the workspace, WebGL2 for the 3D world and for bloom, lens and grain
Capture	Playwright screenshots	JPEG frames piped straight into ffmpeg, no frames written to disk
Chunking	`render_local.py`	300-frame chunks; finished chunks are kept and skipped on the next run; chunks made with older scene files are moved aside automatically
Encoding	ffmpeg, x264	Standard version: two-pass 600 kbit/s (about 27 MB); high-quality version: CRF 18
Fonts	Subset Noto CJK (serif, sans, mono), DejaVu Sans Mono, Lora italic	Bundled with the project so every machine renders the same text
Audio	FLAC	Shipped in parts and joined on first run

Local render package (in `D:\aivideo\music\render_project`): `video.html`, `journey.js`, `world3d.js`, `vis5_data.js`, `fonts/`, `song_part_00-02.bin`, `render_local.py`, `开始渲染.bat` and `说明.txt`. The launcher checks for Python and ffmpeg, installs Playwright if needed, reports which GPU Chrome is using, and resumes from the last finished chunk.

Large-file transfer: files above about 27 MB timed out when copied to your computer, so finished videos were split into 9 MB pieces with a `.bat` that joins them (`copy /b`).

Problems, fixes and lessons

Most late-stage problems were motion problems in the 3D world, not design problems; each was traced to a measurable cause before it was fixed.

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

Problem (latest first)	Root cause	Fix
Lake surface jittered side to side on your GPU	The water was one huge 160,000-unit triangle pair; per-vertex interpolation lost precision across it, and the reflection was distorted by up to 130 px	Each pixel now computes its own point on the water plane from the camera ray (with exact depth); reflection distortion cut to 0.012 with a light blur; mirrored lines dimmed to 35%
Return from the world to the tracks felt abrupt	The return camera followed the river path, so its heading stalled and then spun	A world-space glide with one smooth turn, roll held level, a 2.1 s hand-over; worst rotation jerk fell from 0.067 to 0.019
Raft and boat jumped	Two placement rules switched at a threshold (height 200, time 205 s)	One continuous formula; sideways drift eased in smoothly
Bottom of frame shimmered during the fly-through	Water ripples about 13 units apart while the camera moved about 7.6 units per frame (temporal aliasing)	Wider ripple spacing, ripples fade near the camera, hatching removed, calmer surface near the lens
Camera never reached the lake	The river course was longer than planned	Course scaled to 62% and the camera re-keyed
Seams and dark patches on flat land	A lowered band cut a trench; flat-land normals were wrong	Band limited and blended; normals blended by terrain growth
Cloud renders stopped part-way	Background jobs end after 2 hours	Resumable chunk scripts; local GPU render as the main path
Video copy to your computer timed out	Files around 27 MB exceeded the transfer limit	9 MB pieces plus a join script

Lessons for the next film

- Keep one data source for sound and picture; it removes sync as a class of problem.
- In moving 3D, spacing between repeated lines must be large compared with how far the camera moves in one frame, or it shimmers.

免费分享 · 禁止售卖 · 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free · Not for sale · No paid access of any kind (CC BY-NC-SA 4.0)

- Never draw huge surfaces with per-vertex values; compute them per pixel.
- Give the camera its own smooth path for big moves instead of tying it to an object's path.
- Check motion on the target GPU early: software rendering hid the water jitter that your GPU showed.

Open item: the final title still reads "告诉我，你是怎么作曲的" while the opening question reads "讲讲你是怎样作一首曲子的".

免费分享 • 禁止售卖
FREE TO SHARE • NOT FOR SALE

免费分享 • 禁止售卖 • 禁止以任何名义收费（CC BY-NC-SA 4.0，本项目作者） Shared free • Not for sale • No paid access of any kind (CC BY-NC-SA 4.0)